

JAVA DEVELOPER INTERNSHIP

TASK 1: JAVA BASICS AND OOP CONCEPTS

OBJECTIVE: The objective of this task is to understand the fundamental concepts of Java programming and Object-Oriented Programming (OOP). This task focuses on implementing arrays, loops, conditional statements, encapsulation, inheritance, and polymorphism through simple Java programs. It helps in developing problem-solving skills and gaining practical experience in Java application development.

Task Covered Concepts:

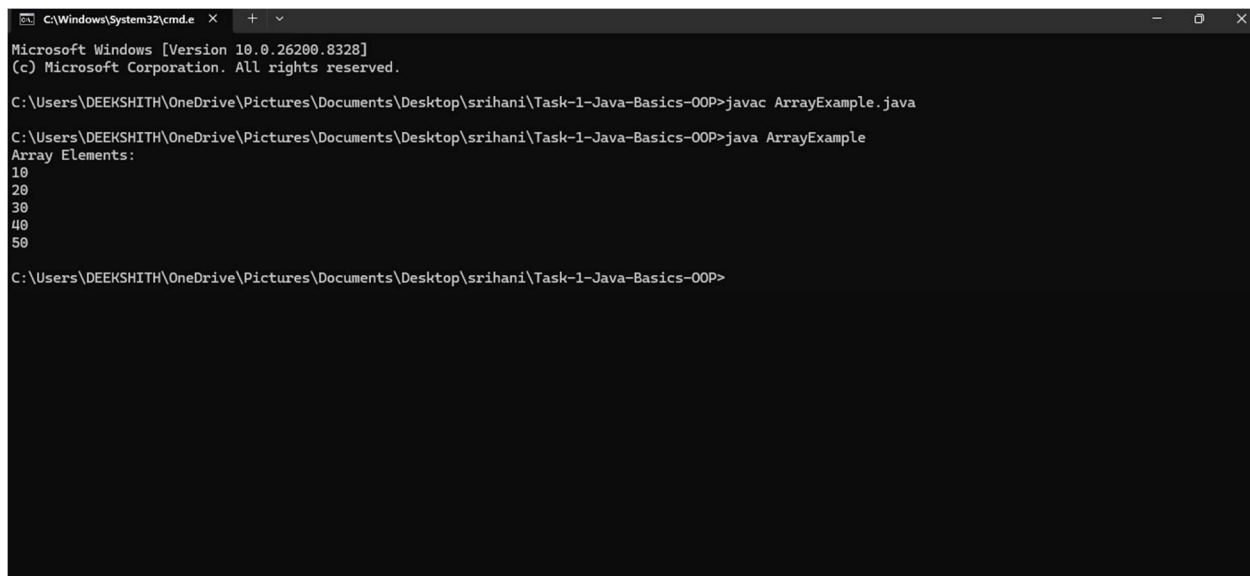
- Arrays
- Loops
- Conditional Statements
- Encapsulation
- Inheritance
- Polymorphism

PROGRAM 1: ARRAYS

Code (ArrayExample.java):

```
public class ArrayExample
{
    public static void main(String[] args)
    {
        int[] numbers = {10, 20, 30, 40, 50};
        System.out.println("Array Elements:");
        for (int i = 0; i < numbers.length; i++)
        {
            System.out.println(numbers[i]);
        }
    }
}
```

Output:



```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>javac ArrayExample.java
C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>java ArrayExample
Array Elements:
10
20
30
40
50
C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>
```

Explanation:

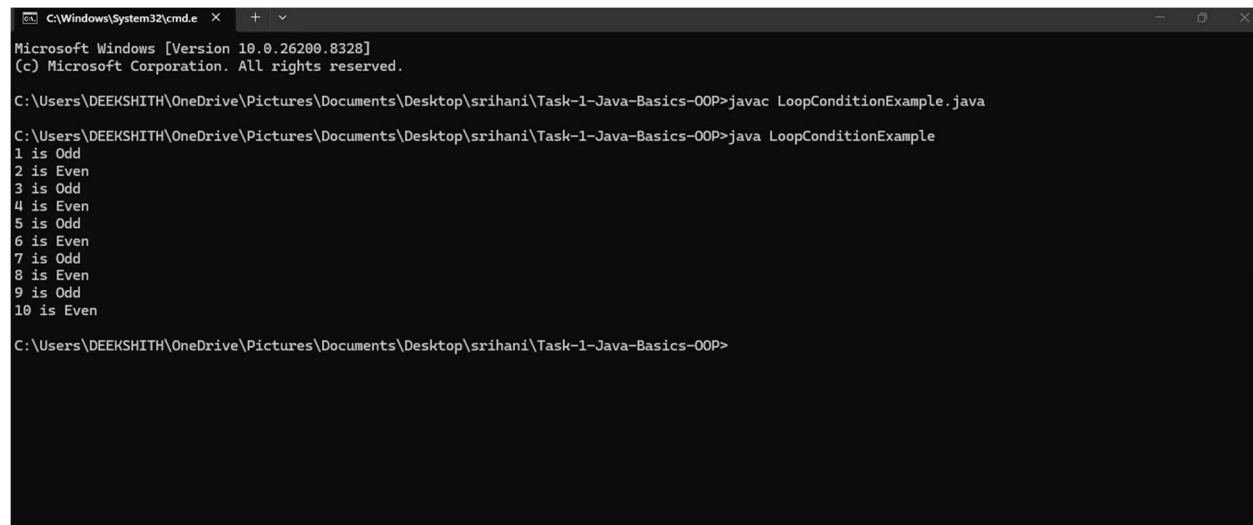
This program demonstrates the use of arrays in Java. An array is used to store multiple integer values, and a loop is used to access and display each element. Arrays help in managing a collection of similar data efficiently.

PROGRAM 2: LOOPS AND CONDITIONS

Code (LoopConditionExample.java):

```
public class LoopConditionExample
{
    public static void main(String[] args)
    {
        for (int i = 1; i <= 10; i++)
        {
            if (i % 2 == 0)
            {
                System.out.println(i + " is Even");
            }
            else
            {
                System.out.println(i + " is Odd");
            }
        }
    }
}
```

Output:



```
C:\Windows\System32\cmd.exe X + v
Microsoft Windows [Version 10.0.26280.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>javac LoopConditionExample.java

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>java LoopConditionExample
1 is Odd
2 is Even
3 is Odd
4 is Even
5 is Odd
6 is Even
7 is Odd
8 is Even
9 is Odd
10 is Even

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>
```

Explanation:

This program demonstrates the use of loops and conditional statements in Java. The for loop is used to iterate through numbers from 1 to 10, while the if-else statement checks whether each number is even or odd.

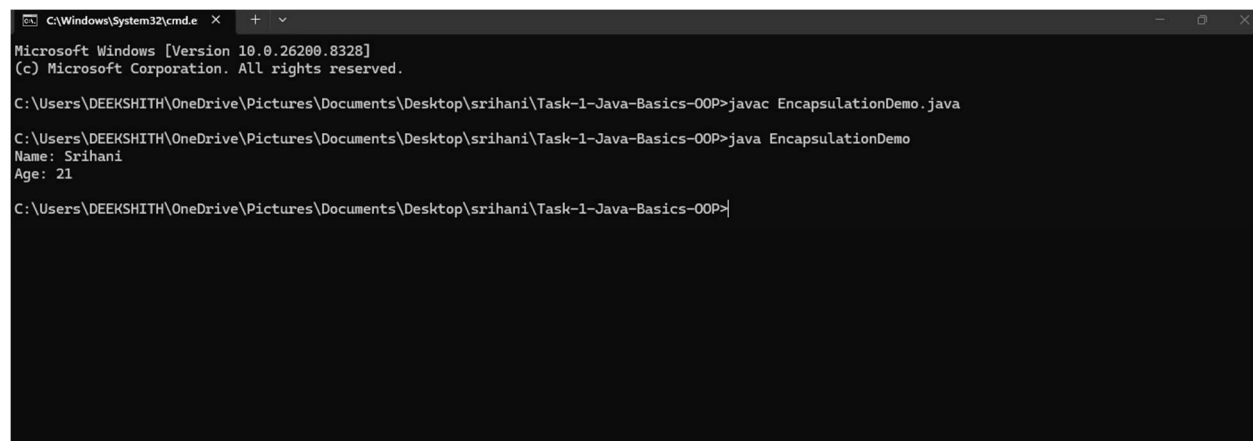
PROGRAM 3: ENCAPSULATION

Code (EncapsulationDemo.java):

```
class Student
{
    private String name;
    private int age;
    public void setName(String name)
    {
        this.name = name;
    }
    public String getName()
    {
        return name;
    }
    public void setAge(int age)
    {
        this.age = age;
    }
    public int getAge()
    {
        return age;
    }
}
```

```
public class EncapsulationDemo
{
    public static void main(String[] args)
    {
        Student student = new Student();
        student.setName("Srihani");
        student.setAge(21);
        System.out.println("Name: " + student.getName());
        System.out.println("Age: " + student.getAge());
    }
}
```

Output:



```
C:\Windows\System32\cmd.exe X + v
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>javac EncapsulationDemo.java

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>java EncapsulationDemo
Name: Srihani
Age: 21

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>|
```

Explanation:

This program demonstrates encapsulation in Java. The data members name and age are declared as private and are accessed through public getter and setter methods. This helps in protecting data and controlling access to class variables.

PROGRAM 4: INHERITANCE AND POLYMORPHISM

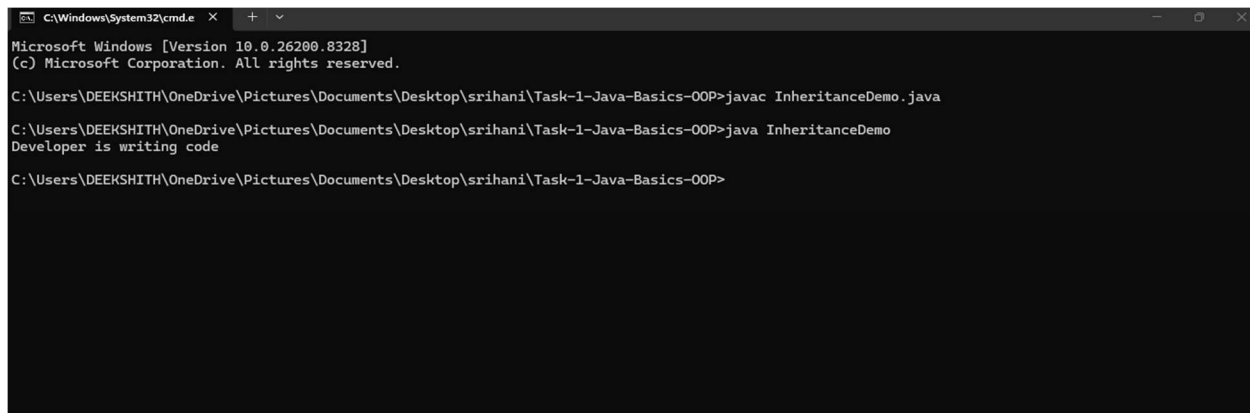
Code (InheritanceDemo.java):

```
class Employee
{
    void work()
    {
        System.out.println("Employee is working");
    }
}

class Developer extends Employee
{
    @Override
    void work()
    {
        System.out.println("Developer is writing code");
    }
}

public class InheritanceDemo
{
    public static void main(String[] args)
    {
        Employee emp = new Developer();
        emp.work();
    }
}
```

Output:



```
C:\Windows\System32\cmd.exe X + v
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>javac InheritanceDemo.java

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>java InheritanceDemo
Developer is writing code

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-1-Java-Basics-OOP>
```

Explanation:

This program demonstrates inheritance and polymorphism in Java. The Developer class inherits the properties and methods of the Employee class. The work() method is overridden in the Developer class, and polymorphism is achieved by using a parent class reference to refer to a child class object.

OOP CONCEPTS USED:

Class:

A class is a blueprint used to create objects. It defines the properties and methods of an object. Classes help organize and structure the program efficiently.

Object:

An object is an instance of a class that can access the class's properties and methods. Objects represent real-world entities in a program.

Encapsulation:

Encapsulation is the process of hiding data using private variables and providing access through getter and setter methods. It improves data security and maintainability.

Inheritance:

Inheritance allows one class to acquire the properties and methods of another class, promoting code reusability. It helps reduce code duplication and supports hierarchical relationships.

Polymorphism:

Polymorphism allows a method to perform different actions depending on the object that calls it. It is achieved through method overriding in Java. This increases flexibility and extensibility in programs.

Method Overriding:

Method overriding occurs when a child class provides a specific implementation of a method already defined in its parent class. It enables runtime polymorphism and customized behavior.

CONCLUSION:

This task provided practical knowledge of Java fundamentals and Object-Oriented Programming concepts. Through the implementation of arrays, loops, conditions, encapsulation, inheritance, and polymorphism, a better understanding of Java programming was achieved. These concepts form the foundation for developing efficient and maintainable Java applications.